VIZUALIZACIJA PODATAKA · VJEŽBA 02 · 2026./27.

Graf raspršenosti u D3.js

Rosište i relativna vlažnost · dvije kvantitativne varijable · po jedna točka za svaki dan.

D3 SCATTERPLOT DATA JOIN

Ishodi vježbe

WHAT

Opisati podatke

Jedan dan kao opažanje; rosište i vlažnost kao dvije mjere.

WHY

Istražiti odnos

Prepoznati oblik povezanosti i izdvojene vrijednosti.

HOW

Izraditi graf

Koristiti SVG kružnice, skale, osi i D3 `.join()`.

> Na kraju ćemo moći ažurirati točke bez stvaranja duplikata.

Zadatak: odnos rosišta i vlažnosti

Temperatura rosišta

`dewPoint` — temperatura pri kojoj zrak, pri odgovarajućim uvjetima, postaje zasićen vlagom.

U datoteci je u °F; prikazat ćemo u °C.

Relativna vlažnost

`humidity` — omjer trenutačne i zasićene količine vodene pare u zraku.

U datoteci kao udio 0–1; prikazat ćemo kao postotak.

Pitanje: vidimo li odnos između tih dviju mjera tijekom 2018.?

Primjer dovršenog grafa

Rosište i relativna vlažnost

365 dnevnih opažanja za 2018. · lokalni meteorološki podatci iz V01 · demonstracija D3 selekcija

Prvih 200 dana | Svih 365 dana | Oboji prema oblačnosti

365 točaka

ENTER 365 | UPDATE 0 | EXIT 0

Otvori graf u zasebnoj kartici ↗

Svaka točka predstavlja jedno dnevno opažanje. Gumbi pokazuju kako D3 ažurira postojeći prikaz.

Podatci iz V01

Prvi zapis u `vrijeme.json`

```
{
  "date": "2018-01-01",
  "dewPoint": -1.67,
  "humidity": 0.54,
  "cloudCover": 0.02
}
```

Izvorni prvi zapis (odabrani atributi).

365 dnevnih opažanja

Opažanje: **jedan dan** u 2018.

`date` : vremenski identifikator.

`dewPoint` , `humidity` : kvantitativni atributi.

P02 → V02: najprije tumačenje jedinica i semantike.

Zašto graf raspršenosti?

V01: vremenski tijek

Jedan atribut mjerimo kroz slijed dana.

datum → **temperatura**

Linija naglašava kronološki redoslijed.

V02: odnos dvaju atributa

Za svaki dan uspoređujemo dvije mjere.

rosište ↔ **vlažnost**

Položaj točke određuju vrijednosti na dvjema osima.

Ne spajamo točke linijom: redoslijed dana nije primarni cilj prikaza.

Početni projekt

Datoteke

```
index.html
stil.css
scatter.js      ← dopuniti
vrijeme.json    ← 365 zapisa
d3.js           ← lokalni D3
```

Pokretanje

1. Preuzmite ZIP početnog projekta.
2. Otvorite mapu u VS Codeu.
3. Pokrenite **Live Server**.
4. Otvorite **Developer Tools** → **Console**.

Važno: `d3.json()` zahtijeva HTTP poslužitelj; izravno `file://` otvaranje može biti blokirano.

Redoslijed izrade

1. Podatci
2. Dimenzije
3. SVG
4. Skale
5. Točke
6. Osi
7. Provjera

Ulaz

Čitamo JSON i definiramo koje nas vrijednosti zanimaju.

Mapiranje

Podatkovne vrijednosti pretvaramo u koordinate.

Prikaz

Stvaramo i ažuriramo SVG elemente.

> Razlika u odnosu na V01: ne crtamo putanju linije, nego po jednu kružnicu za svaki zapis.

1. Učitavanje podataka

```
async function crtajScatter() {
  const dataset = await d3.json("vrijeme.json");
  console.log(dataset.length);
  console.log(dataset[0]);
  // daljnji kod dolazi ovdje
}
crtajScatter().catch(console.error);
```

> **U konzoli provjerite: 365 zapisa i prvi dnevni zapis s poljima `dewPoint` i `humidity`.**

1. Funkcije za dohvat vrijednosti

```
const xAccessor = d => (d.dewPoint - 32) * 5 / 9;
const yAccessor = d => d.humidity * 100;
```

Rosište

−18,7 °C

Prvi zapis: −1,67 °F → približno −18,7 °C.

Relativna vlažnost

54 %

Prvi zapis: udio 0,54 → 54 %.

Zašto je bolje pretvorbu jedinica smjestiti u accessor nego je ponavljati u svakom `attr()` pozivu?

2. Dimenzije grafa

```
const dimenzije = {
  sirina: 820,
  visina: 540,
  margine: { top: 35, right: 30, bottom: 70, left: 82 }
};

dimenzije.grSirina = dimenzije.sirina
  - dimenzije.margine.left - dimenzije.margine.right;
dimenzije.grVisina = dimenzije.visina
  - dimenzije.margine.top - dimenzije.margine.bottom;
```

Margine rezerviramo za osi i oznake; `grSirina` i `grVisina` su unutarnje dimenzije prikaza.

3. SVG i grupa za podatke

```
const okvir = d3.select("#okvir").append("svg")
  .attr("viewBox", `0 0 ${dimenzije.sirina} ${dimenzije.visina}`);

const granice = okvir.append("g")
  .attr("transform", `translate(
    ${dimenzije.margine.left},
    ${dimenzije.margine.top}
  )`);
```

`svg`

Vanjski koordinatni sustav i prostor za osi.

`g`

Unutarnji koordinatni sustav — pomaknut za margine.

4. Skala osi x

```
const xSkala = d3.scaleLinear()
  .domain(d3.extent(dataset, xAccessor))
  .range([0, dimenzije.grSirina])
  .nice();

console.log(xSkala.domain());
```

Domena

Raspon rosišta iz podataka, u **°C**.

Raspon

Horizontalni prostor: **od 0 do grSirina** piksela.

4. Skala osi y

> **Napišite y-skalu samostalno: koristite relativnu vlažnost i pritom obrnite raspon piksela.**

```
const ySkala = d3.scaleLinear()
  .domain(d3.extent(dataset, yAccessor))
  .range([dimenzije.grVisina, 0])
  .nice();
```

U SVG-u koordinata y raste prema dolje. Zato veća vlažnost treba imati **manju** y-koordinatu.

Provjera domena i metoda .nice()

```
console.log("x", xSkala.domain());
console.log("y", ySkala.domain());
```

`d3.extent()`

Pronalaženje najmanje i najveće vrijednosti atributa.

`.nice()`

Proširivanje domene na preglednije granice.

`.domain()`

Čitanje trenutačne domene postojeće skale.

Granice domene nisu nužno vrijednosti nekog konkretnog dana.

5. Jedna SVG kružnica

```
granice.append("circle")
  .attr("cx", dimenzije.grSirina / 2)
  .attr("cy", dimenzije.grVisina / 2)
  .attr("r", 5)
  .attr("fill", "#168b7c");
```

`cx`

Horizontalna pozicija središta.

`cy`

Vertikalna pozicija središta.

`r`

Polumjer kružnice u pikselima.

Nakon provjere uklonite probni krug.

5. Crtanje točaka petljom

```
dataset.forEach(d => {
  granice.append("circle")
    .attr("cx", xSkala(xAccessor(d)))
    .attr("cy", ySkala(yAccessor(d)))
    .attr("r", 3);
});
```

Radi za prvi prikaz

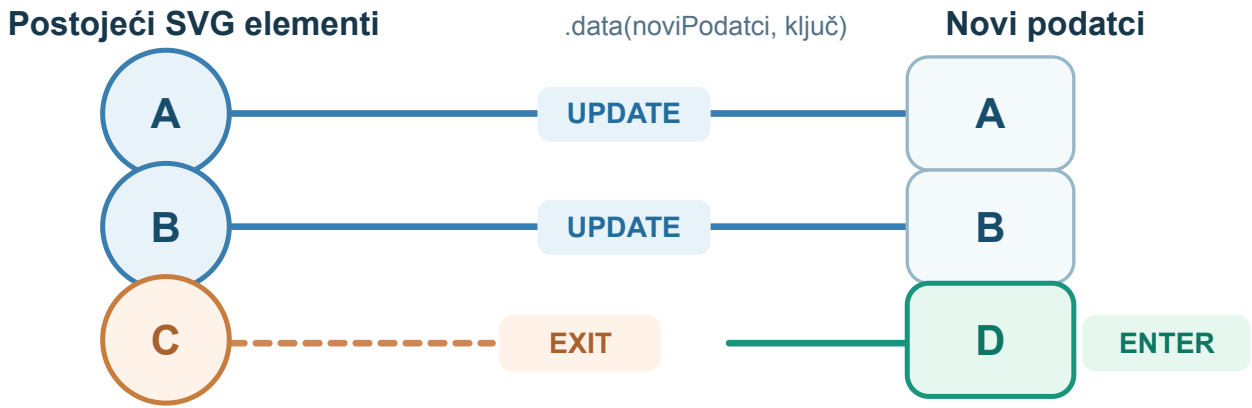
Jedno opažanje → jedan SVG krug.

Problem pri ažuriranju

Svaki novi poziv ponovno dodaje krugove, čak i za postojeća opažanja.

Što će se dogoditi ako isti kod izvršimo dvaput?

Povezivanje podataka s SVG elementima



UPDATE: A, B **EXIT: C** **ENTER: D**

`.data()` određuje što je novo, postojeće ili suvišno. **Samo povezivanje još ne crta niti briše SVG elemente.**

Selekcija i povezivanje podataka

```
const vezani = granice.selectAll("circle.tocka")
  .data(dataset, d => d.date);

console.log("update", vezani.size());
console.log("enter", vezani.enter().size());
console.log("exit", vezani.exit().size());
```

`selectAll()`

Odabire postojeće kružnice klase `tocka`.

`data()`

Povezuje opažanja s elementima prema ključu `date` i vraća **update selekciju**.

Ako prije toga nije bilo točaka, koliko iznose enter, update i exit?

Enter, update i exit

Prije: 200 točaka → novi skup: 365 opažanja

ENTER 165 | UPDATE 200 | EXIT 0

Prije: 365 točaka → novi skup: 200 opažanja

ENTER 0 | UPDATE 200 | EXIT 165

enter → stvori element | update → prilagodi element | exit → ukloni element

Pretpostavka: ista opažanja imaju stabilni ključ `date`. Kod potpunog prvog crtanja 365 točaka imamo enter = 365, update = 0, exit = 0.

Metoda .enter()

```
const vezani = granice.selectAll("circle.tocka")
  .data(dataset, d => d.date);

vezani.enter().append("circle")
  .attr("class", "tocka")
  .attr("cx", d => xSkala(xAccessor(d)))
  .attr("cy", d => ySkala(yAccessor(d)))
  .attr("r", 3);
```

`.enter()` vraća podatke kojima nedostaje SVG element. `.append()` stvara krugove **samo za te podatke**.

Ako ponovimo ovaj kod s istim podatcima, novih elemenata nema; ali atributi već postojećih elemenata neće se automatski osvježiti.

Ažuriranje pomoću .merge()

```
const vezani = granice.selectAll("circle.tocka")
  .data(podaci, d => d.date);
const novi = vezani.enter().append("circle")
  .attr("class", "tocka");

novi.merge(vezani)
  .attr("cx", d => xSkala(xAccessor(d)))
  .attr("cy", d => ySkala(yAccessor(d)))
  .attr("r", 4);
vezani.exit().remove();
```

novi → enter | **postojeći → update** | **exit → remove()**

`.merge()` spaja **enter i update** kako bismo svim zadržanim točkama postavili iste atribute. **Ne uklanja** exit elemente.

Metoda .join()

```
granice.selectAll("circle.tocka")
  .data(podaci, d => d.date)
  .join("circle")
  .attr("class", "tocka")
  .attr("cx", d => xSkala(xAccessor(d)))
  .attr("cy", d => ySkala(yAccessor(d)))
  .attr("r", 4);
```

ENTER

Stvara nove kružnice.

UPDATE

Zadržava postojeće kružnice.

EXIT

Uklanja kružnice bez odgovarajućeg podatka.

`.join("circle")` u ovom slučaju zamjenjuje `enter().append()`, `merge()` i `exit().remove()`. Atribute zatim postavljamo na vraćenoj selekciji.

Stabilan ključ pri ažuriranju

```
granice.selectAll("circle")
  .data(podaci, d => d.date)
  .join("circle")
  .attr("cx", d => xSkala(xAccessor(d)))
  .attr("cy", d => ySkala(yAccessor(d)))
  .attr("r", 4);
```

Bez ključa

D3 prema zadanim pravilima uparuje elemente s podatcima po redoslijedu.

S ključem datuma

Prati identitet dnevnog opažanja i kad se niz promijeni.

Ovdje `date` vrijedi kao identifikator jer imamo jedno opažanje za svaki dan.

Provjera ažuriranja: 200 → 365 → 200

```
function crtajTocke(podaci) {
  const vezani = granice.selectAll("circle.tocka")
    .data(podaci, d => d.date);

  console.log({
    enter: vezani.enter().size(),
    update: vezani.size(),
    exit: vezani.exit().size()
  });
  vezani.join("circle")
    .attr("class", "tocka")
    .attr("cx", d => xSkala(xAccessor(d)))
    .attr("cy", d => ySkala(yAccessor(d)))
    .attr("r", 4);
}
```

Pozovite funkciju redom za `dataset.slice(0,200)`, `dataset` i ponovno prvih 200 zapisa. Provjerite brojeve u konzoli.

6. Osi grafa

```
const xOs = granice.append("g")
  .attr("transform", `translate(0,${dimenzije.grVisina})`)
  .call(d3.axisBottom(xSkala).ticks(8));

const yOs = granice.append("g")
  .call(d3.axisLeft(ySkala).ticks(7));
```

Osi su dva SVG `g` elementa. Donju pomičemo na visinu unutarnjeg grafa, a lijeva već počinje na x = 0.

6. Oznake i jedinice

```
xOs.append("text")
  .attr("x", dimenzije.grSirina / 2)
  .attr("y", 54)
  .attr("text-anchor", "middle")
  .text("Temperatura rosišta (°C)");

yOs.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -dimenzije.grVisina / 2)
  .attr("y", -60)
  .attr("text-anchor", "middle")
  .text("Relativna vlažnost (%)");
```

Grupe osi su na prethodnom slajdu pohranjene u varijable `xOs` i `yOs`. Po potrebi dodajte `fill` i font-size.

Tumačenje raspršenog grafa

Oblik odnosa

Promatrajte raste li vlažnost uz rosište ili postoji drukčiji obrazac.

Izdvojene vrijednosti

Postoje li točke ili skupine točaka koje se značajno razlikuju od većine?

Ograničenja

Jedna godina, dnevna agregacija, moguća druga meteorološka obilježja.

U konkretnom skupu je Pearsonov **r ≈ 0,55** za rosište u °F (ili °C) i relativnu vlažnost; to **ne dokazuje uzrok**.

Izračunato iz 365 dnevnih zapisa; koeficijent se ne računa D3 kodom na ovoj vježbi.

Dodatni zadatak: oblačnost

Treći atribut

`cloudCover` je udio naoblake između 0 i 1.

Već imamo x i y položaj; oblačnost ćemo prikazati **bojom točke**.

Provjera iz P02

Oblačnost je **kvantitativni** atribut sa sekvencijalnim rasponom.

Pogodna je jednosmjerna skala boje od svjetlije prema tamnijoj.

Zadatak: bez promjene koordinata, povežite vrijednost oblačnosti s bojom svake točke.

Skala boje za oblačnost

```
const skalaBoje = d3.scaleSequential(d3.interpolateBlues)
  .domain([0, 1]);

granice.selectAll("circle.tocka")
  .attr("fill", d => skalaBoje(d.cloudCover));
```

Domena

Vrijednost `cloudCover` od **0 do 1** (0–100 % pokrivenosti oblacima).



0 % 50 % 100 %

Raspon boja

`d3.interpolateBlues` daje niz nijansi od svijetloplave do tamnoplave.

Kao i položajna skala, skala boje preslikava **vrijednost u vizualni atribut**. Za tumačenje mora imati legendu.

Samostalni zadatci

A · OSNOVNI

Filtriranje opažanja

Prikažite samo dane s relativnom vlažnošću od najmanje 70 %. Vratite sva opažanja istom funkcijom `crtajTocke()`.

Provjerite broj krugova i status enter/update/exit.

B · SREDNJI

Identitet opažanja

Promijenite redoslijed odabranih podataka sortiranjem po rosištu. Provjerite ostaju li isti podatci vezani uz iste datume.

Objasnite razliku između uparivanja po indeksu i ključu `date`.

C · DODATNI

Isticanje izdvojene vrijednosti

Pronađite dan s najvećim rosištem te mu dodajte razlikovni obrub i naslov s datumom.

Broj točaka mora ostati nepromijenjen.

Zadatke rješavajte bez novih SVG krugova izvan `.join()`. Detalji i kriteriji provjere nalaze se uz upute V02.

Sažetak V02

WHAT

365 dana

Rosište (°C), vlažnost (%) i opcionalno oblačnost.

WHY

Odnos mjera

Traženje povezanosti, raspršenja i iznimaka.

HOW

D3 graf

Linearne skale + SVG krugovi + `.data().join()` + osi.

> Ključna razlika u odnosu na V01: podatci su povezani s mnogim SVG elementima, koji se mogu pojedinačno ažurirati.

D3 dokumentacija

d3.json() ↗
Učitavanje JSON podataka.

d3.extent() ↗
Najmanja i najveća vrijednost.

scaleLinear(), nice() ↗
Linearno preslikavanje.

selection.data() ↗
Povezivanje podataka s elementima.

selection.join() ↗
Dodavanje, ažuriranje i uklanjanje.

scaleSequential() ↗
Preslikavanje brojeva u boje.

Kod za vježbu i zadaci nalaze se uz prezentaciju na portalu. Izvorni V02 je polazište sadržaja; primjeri su prilagođeni D3 v7 i V01.